

Lab 03 · Cuando el trámite no procede

Curso de Spring Boot · Servicio de Impuestos Internos · 2026

60 minutos · Spring Boot 4.1.0 · Java 25 (Temurin)

Resumen

Que un error también es una respuesta, y que hay que escribirla: un 404 que dice qué no se encontró, un 400 que nombra los campos que venían mal, y un 500 que le cuenta al público lo justo mientras deja en el registro interno todo lo demás.

Índice

1. Antes de empezar	2
1.1. Qué vas a lograr	2
1.2. Qué necesitas tener listo	2
1.3. Cómo copiar el código de esta guía	2
1.4. La puesta a punto	3
2. El caso	3
2.1. El papel que explica por qué, que es la metáfora de este laboratorio	3
3. Los pasos	3
3.1. Paso 1 · Mirar lo que sale hoy	3
3.2. Paso 2 · Una excepción que dice lo que pasó	4
3.3. Paso 3 · El traductor	5
3.4. Paso 4 · Lo que manda mal quien llama	8
3.5. Paso 5 · La red de seguridad, y lo que se calla	10
4. Lo que aprendiste	12
5. Para profundizar	12
6. Antes de cerrar	12

1 Antes de empezar

1.1 Qué vas a lograr

En el Lab 01 devolviste un 404 con el cuerpo vacío. Servía para las máquinas y no para las personas: no decía **qué** no se encontró.

Hoy vas a hacer que cada error salga por la ventanilla con la misma forma: un cuerpo con mensaje, código y momento. Vas a ver un 400 que nombra **campo por campo** qué venía mal. Y vas a montar una red de seguridad para lo que nadie previó, aprendiendo a la vez la regla más incómoda del laboratorio: **qué NO se le cuenta a quien llama**.

1.2 Qué necesitas tener listo

Requisito	Cómo lo compruebas	Qué tiene que salir
Los labs 00 a 02 hechos	Sabes crear endpoints y pedir dependencias	—
Estar en la carpeta del lab	<code>cd labs/lab-03-errores/practica</code>	El cd no da error
Dos terminales	Una con la app, otra para los <code>curl</code>	—

1.3 Cómo copiar el código de esta guía

Al copiar de un PDF se pierden los espacios del principio de cada línea, y a veces una línea larga se parte en dos. Con Java casi nunca importa; si una línea se parte, el editor te la marca y basta con unirlos. El código completo está en `labs/lab-03-errores/solucion/`.

1.4 La puesta a punto

```
cd labs/lab-03-errores/practica
```

Esta aplicación escucha en el puerto **8085**. Ctrl+C para pararla, y hay que reiniciarla después de cada cambio.

2 El caso

En la oficina de la DGT, la mitad de lo que llega a la ventanilla **no procede**: piden un expediente que no existe, entregan un formulario a medias, o pasa algo que nadie había previsto.

2.1 El papel que explica por qué, que es la metáfora de este laboratorio

Un error no es un portazo: es un papel.

Cuando un trámite no procede, una oficina que funciona no te dice «vuelva mañana» ni te cierra la ventanilla en la cara. Te entrega **un papel** que dice tres cosas: qué pasó, con qué código, y cuándo. Con ese papel puedes hacer algo — corregir lo que falta, o reclamar citando el código.

Y hay una cuarta cosa que ese papel **no** dice, y es igual de importante: **no cuenta los problemas internos de la oficina**. Si se cayó el sistema del sótano, el papel dice «ha ocurrido un problema, inténtelo más tarde». Lo que se cayó exactamente, y por qué, queda escrito en el **registro interno** — donde lo lee quien puede arreglarlo, y no quien está en la cola.

Esa es la separación que monta este laboratorio: **lo que se dice y lo que se apunta**.

3 Los pasos

3.1 Paso 1 · Mirar lo que sale hoy

Qué vamos a hacer

Pedir un producto que no existe y leer, con calma, lo que la aplicación contesta de fábrica.

Para entenderlo mejor

Ir a la ventanilla y pedir un expediente inventado, para ver qué papel te dan **antes** de que nadie haya diseñado ninguno.

El problema

Spring no se queda callado cuando algo falla: contesta algo. El problema es que ese algo lo eligió él, no tú, y casi nunca es lo que quieres que vea quien llama.

Se corre

```
./mvnw spring-boot:run
```

y desde otra terminal:

```
curl -i localhost:8085/productos/99
```

Lo que vas a ver

Un error genérico, sin nada que identifique **qué** producto faltaba. Y lo peor no es lo que dice: es que **la forma de ese cuerpo la decidió el framework**. El día que cambies de versión, o de framework, quien consuma tu API tiene que cambiar con él.

Vas bien si...

Obtuviste una respuesta de error a /productos/99 y puedes decir qué le falta: no nombra el producto que buscabas.

Si te atascas

1 · Port 8085 was already in use

Tienes la aplicación arrancada en otra terminal. Ciérrala con Ctrl+C, o:

```
lsof -ti:8085 | xargs kill -9
```

2 · curl no muestra el código de estado.

Falta el -i.

3.2 Paso 2 · Una excepción que dice lo que pasó

Qué vamos a hacer

Escribir una excepción propia y lanzarla cuando el producto no aparezca.

Para entenderlo mejor

Que quien atiende **levante la mano y diga en voz alta qué ha pasado**, en vez de improvisar una respuesta. Todavía no está el papel; lo que hay es un aviso claro de que este caso no procede, y por qué.

El problema

Un if que devuelve una respuesta de error dentro del método mezcla dos cosas: la lógica de buscar el producto y la de contestar por HTTP. Y si tres endpoints buscan productos, la misma comprobación aparece tres veces.

La alternativa, y por qué no

- **Devolver null** y que quien llama lo compruebe. Es la fuente clásica del NullPointerException: basta que uno de los que llaman se olvide.
- **Devolver optional** y decidir en cada sitio. Es correcto y se usa mucho —el Lab 02 lo hace—, pero obliga a repetir la decisión en cada endpoint.
- **Lanzar una excepción propia**, que es lo de aquí: la comprobación va una vez, donde se busca, y la traducción a HTTP va una vez, en otro sitio. Cada cosa en su capa.

Y la excepción es **propia**, no una genérica: RuntimeException diría «algo pasó»; ProductoNoEncontradoException dice qué pasó, y permite tratarla distinto que a las demás.

Se pega

Archivo **nuevo** practica/src/main/java/cl/dgt/errores/exceptions/ProductoNoEncontradoException.java:

```
package cl.dgt.errores.exceptions;

public class ProductoNoEncontradoException extends RuntimeException {

    public ProductoNoEncontradoException(Long id) {
        super("No existe el producto con id " + id + ".");
    }
}
```

Y en el controlador, el método que la lanza:

```
@GetMapping("/{id}")
public Producto porId(@PathVariable Long id) {
    return base.stream()
        .filter(p -> p.id().equals(id))
        .findFirst()
        .orElseThrow(() -> new ProductoNoEncontradoException(id));
}
```

Fíjate en el `.orElseThrow(...)`: **el caso de que no exista se resuelve donde se busca**, no más adelante.

Se corre

```
curl -i localhost:8085/productos/99
```

Lo que vas a ver

Un **500**. Y está bien que sea 500 por ahora: lanzaste una excepción que nadie está traduciendo, así que para Spring es una avería del servidor.

Ese 500 es el problema del paso siguiente.

Vas bien si...

/productos/99 devuelve 500, y en la consola del servidor aparece tu `ProductoNoEncontradoException` con el mensaje que escribiste.

Si te atascas

1 · Sigue saliendo lo mismo que en el paso 1.

No reiniciaste la aplicación, o el método que editaste no es el que atiende `/productos/{id}`.

2 · cannot find symbol: class ProductoNoEncontradoException

Falta el import en el controlador, o el archivo está fuera de `src/main/java/cl/dgt/errores/exceptions/`.

3.3 Paso 3 · El traductor

Qué vamos a hacer

Escribir el sitio único donde las excepciones se convierten en respuestas HTTP con forma.

Para entenderlo mejor

La mesa de informaciones. Quien atiende levanta la mano; alguien en la mesa de informaciones recoge ese aviso y **redacta el papel**: con su texto, su código y su fecha. La persona de la ventanilla no redacta papeles, y la mesa de informaciones no atiende trámites.

El problema

Sin traductor, cada endpoint tendría que capturar sus excepciones y armar su propia respuesta. Con diez endpoints eso son diez formatos de error ligeramente distintos, y quien consume tu API tiene que aprenderse los diez.

La alternativa, y por qué no

- **Un try/catch en cada método.** Funciona y se repite. Un formato nuevo obliga a tocar todos.
- **@ResponseStatus sobre la clase de la excepción.** Es una línea y da el código correcto — pero **no da cuerpo**: vuelves al 404 vacío del Lab 01.
- **@RestControllerAdvice**, que es lo de aquí: un sitio, todos los controladores, y control total sobre el cuerpo.

Se pega

Archivo **nuevo** practica/src/main/java/cl/dgt/errores/dto/ErrorRespuesta.java — el cuerpo que va a tener **todo** error de esta aplicación:

```
package cl.dgt.errores.dto;

import com.fasterxml.jackson.annotation.JsonInclude;

import java.time.Instant;
import java.util.Map;

@JsonInclude(JsonInclude.Include.NON_NULL)
public record ErrorRespuesta(String mensaje, int codigo, Instant timestamp,
    ↪ Map<String, String> campos) {

    public static ErrorRespuesta de(String mensaje, int codigo) {
        return new ErrorRespuesta(mensaje, codigo, Instant.now(), null);
    }
}
```

Ese @JsonInclude(NON_NULL) es el que hace que el campo campos **no salga** cuando está vacío. Sin él, todos los errores llevarían un "campos": null que no significa nada.

Archivo **nuevo** practica/src/main/java/cl/dgt/errores/exceptions/ManejadorDeErrores.java, por ahora solo con el primer manejador:

```
package cl.dgt.errores.exceptions;
```

```
import cl.dgt.errores.dto.ErrorRespuesta;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class ManejadorDeErrores {

    @ExceptionHandler(ProductoNoEncontradoException.class)
    public ResponseEntity<ErrorRespuesta> noEncontrado(ProductoNoEncontradoException
    ↪ e) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(ErrorRespuesta.de(e.getMessage(), 404));
    }
}
```

Se corre

```
curl -i localhost:8085/productos/99
```

Lo que vas a ver

```
HTTP/1.1 404
{"mensaje":"No existe el producto con id
↪ 99.", "codigo":404, "timestamp":"2026-08-29T03:39:10.053037Z"}
```

404, y con cuerpo. Dice qué producto faltaba, con qué código y cuándo. Con ese papel, quien llama puede hacer algo.

El timestamp cambia en cada llamada. El tuyo no va a coincidir con el de esta guía, y está bien: lo que tiene que coincidir es el mensaje y el código.

Vas bien si...

/productos/99 devuelve 404 —ya no 500— con las tres claves en el cuerpo, y /productos/2 sigue devolviendo el producto normalmente.

Si te atascas

1 · Sigue saliendo 500.

El manejador no se está aplicando. Comprueba tres cosas, en este orden: que la clase tenga `@RestControllerAdvice`; que el método tenga `@ExceptionHandler(ProductoNoEncontradoException.class)` con **esa** excepción y no otra; y que la clase esté bajo `cl.dgt.errores`.

2 · Sale 404 pero el cuerpo está vacío.

Pusiste `@ResponseStatus` en la excepción en vez del manejador, o el manejador devuelve

```
ResponseEntity sin .body(...).
```

3 · Sale un "campos": null en el cuerpo.

Falta el `@JsonInclude(JsonInclude.Include.NON_NULL)` encima del record.

3.4 Paso 4 · Lo que manda mal quien llama

Qué vamos a hacer

Validar el cuerpo de un POST y devolver un 400 que diga **qué campo** venía mal.

Para entenderlo mejor

El formulario incompleto. No se rechaza con un «está mal»: se devuelve **señalando las casillas** —«el nombre está vacío», «el precio no puede ser negativo»— para que se pueda corregir de una vez y no a base de intentos.

El problema

Comprobar los campos a mano dentro del método significa un `if` por campo, un mensaje por `if`, y la validación repetida en cada endpoint que reciba lo mismo. Y casi siempre se devuelve solo el primer error, así que quien llama corrige uno, vuelve a mandar, y descubre el siguiente.

La alternativa, y por qué no

Los `if` a mano funcionan y no dependen de nada. Se descartan porque las reglas quedan lejos del dato que describen: leyendo el objeto no sabes qué es válido. Con las anotaciones, **la regla vive pegada al campo**, y quien lea el record sabe qué se espera.

Y hay una trampa que hay que decir en voz alta: las anotaciones **no hacen nada por sí solas**. Sin `@Valid` en el parámetro del controlador, están decorando.

Se pega

Archivo **nuevo** `practica/src/main/java/cl/dgt/errores/dto/ProductoNuevoDto.java`:

```
package cl.dgt.errores.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Positive;

public record ProductoNuevoDto(
    @NotBlank(message = "el nombre es obligatorio") String nombre,
    @Positive(message = "el precio debe ser mayor que cero") int precio) {
}
```

El método que lo recibe, **con su `@Valid`**:

```
@PostMapping
public ResponseEntity<Producto> crear(@Valid @RequestBody ProductoNuevoDto nuevo) {
    Producto producto = new Producto(siguienteId.getAndIncrement(), nuevo.nombre(),
    ↪ nuevo.precio());
    base.add(producto);
}
```

```

    return ResponseEntity.status(HttpStatus.CREATED).body(producto);
}

```

Y en el manejador, el traductor de los errores de validación:

```

@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<ErrorRespuesta> validacion(MethodArgumentNotValidException e) {
    Map<String, String> campos = new HashMap<>();
    e.getBindingResult().getFieldErrors()
        .forEach(error -> campos.put(error.getField(), error.getDefaultMessage()));

    ErrorRespuesta cuerpo = new ErrorRespuesta(
        "Hay datos inválidos en la petición.", 400, Instant.now(), campos);
    return ResponseEntity.badRequest().body(cuerpo);
}

```

Se corre

```

curl -i -X POST localhost:8085/productos \
  -H 'Content-Type: application/json' \
  -d '{"nombre":"","precio":-5}'

```

Lo que vas a ver

```

HTTP/1.1 400
{"mensaje":"Hay datos inválidos en la petición","codigo":400,
 "timestamp":"2026-08-29T03:39:10.122871Z",
 "campos":{"precio":"el precio debe ser mayor que cero","nombre":"el nombre es
↪ obligatorio"}}

```

Los dos errores a la vez, cada uno con el nombre de su campo. Quien llama corrige de una vez.

Y con un cuerpo correcto:

```

curl -i -X POST localhost:8085/productos \
  -H 'Content-Type: application/json' \
  -d '{"nombre":"Grapadora","precio":5900}'

```

```

HTTP/1.1 201
{"id":4,"nombre":"Grapadora","precio":5900}

```

El orden de los campos dentro de campos puede salirte al revés. Se construye con un mapa sin orden garantizado. Lo que importa es que estén los dos.

Vas bien si...

El POST con datos malos da **400** y nombra los dos campos; el POST con datos buenos da **201** y devuelve el producto con el id que puso el servidor.

Si te atascas

1 · Devuelve 201 y acepta la basura.

```
HTTP/1.1 201
{"id":4,"nombre":"","precio":-5}
```

Éste es el error importante del paso, porque no falla: **acepta**. Falta el `@Valid` delante del `@RequestBody`. Las anotaciones del record no se aplican solas; `@Valid` es lo que las enciende.

2 · Devuelve 400 pero sin el detalle de los campos.

Falta el manejador de `MethodArgumentNotValidException`, así que está contestando el 400 genérico de Spring en vez del tuyo.

3 · `cannot find symbol: class NotBlank`

Faltan los `import` de `jakarta.validation.constraints`.

3.5 Paso 5 · La red de seguridad, y lo que se calla

Qué vamos a hacer

Añadir un manejador para **todo lo demás**, y provocar a propósito un error que nadie escribió.

Para entenderlo mejor

El aviso general de la oficina: «*ha ocurrido un problema, inténtelo más tarde*». Es lo que se le dice al público cuando lo que se rompió es de dentro. Y a la vez, **en el registro interno queda todo**: qué se rompió, dónde y cuándo.

Las dos cosas a la vez, y no una: si al público se le cuenta el detalle técnico, se le está dando un mapa del edificio a cualquiera que pregunte.

El problema

Por muchos casos que preveas, siempre queda uno. Y lo que sale entonces —una traza de Java, el nombre de una clase interna, la consulta que falló— es información que no debe salir por la ventanilla.

La alternativa, y por qué no

- **No poner red de seguridad** y dejar que salga el error por defecto. Es lo que filtra información sin querer.
- **Devolver el mensaje de la excepción** al que llama. Cómodo para depurar y peligroso: el texto de una excepción suele traer rutas, nombres de tabla o de columna.
- **Un mensaje genérico y el detalle al registro**, que es lo de aquí. Quien llama sabe que falló; quien opera sabe por qué.

Se pega

En el manejador, el último método:

```
@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorRespuesta> todoLoDemas(Exception e) {
```

```

log.error("Error no previsto atendiendo una petición", e);
return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
    .body(ErrorRespuesta.de("Ocurrió un error inesperado. Inténtalo más
        ↪ tarde.", 500));
}

```

Y el endpoint que va a romperse, en el controlador — con la división que nadie protegió:

```

@GetMapping("/{id}/cuota")
public int cuota(@PathVariable Long id, @RequestParam int cuotas) {
    Producto producto = porId(id);
    return producto.precio() / cuotas;
}

```

Se corre

```
curl -i "localhost:8085/productos/1/cuota?cuotas=0"
```

Lo que vas a ver

Lo que recibe quien llama:

```

HTTP/1.1 500
{"mensaje":"Ocurrió un error inesperado. Inténtalo más tarde.", "codigo":500,
 "timestamp":"2026-08-29T03:39:10.151891Z"}

```

Y lo que queda en **la consola del servidor**, que es la otra mitad:

```

ERROR ... c.d.e.exceptions.ManejadorDeErrores : Error no previsto atendiendo una
    ↪ petición
java.lang.ArithmeticException: / by zero

```

Mira las dos cosas juntas. Quien llama sabe que falló y no sabe nada más. Quien opera tiene el nombre exacto de la excepción y la línea. Nadie escribió el caso «dividir por cero»: la red lo recogió igual.

Vas bien si...

El curl devuelve 500 con el mensaje genérico, **y** en la terminal donde corre la aplicación aparece `java.lang.ArithmeticException: / by zero`. Si solo ves una de las dos, falta algo.

Si te atascas

1 · El 500 sale con el mensaje técnico dentro.

Estás devolviendo `e.getMessage()` en el cuerpo. Eso es justo lo que este paso enseña a no hacer: el mensaje al registro, el genérico al cuerpo.

2 · No aparece nada en la consola del servidor.

Falta el `log.error(...)`, o falta el logger. Sin eso el error desaparece del todo, que es peor que el problema original.

3 · El manejador de Exception se traga también los 404.

Spring elige siempre el manejador **más específico** que encaje, así que el de `ProductoNoEncontradoException` gana sobre el de `Exception`. Si tus 404 se volvieron 500, es que borraste el manejador específico.

4 Lo que aprendiste

1 · Un error es una respuesta, y hay que diseñarla.

Si no la escribes tú, la escribe el framework — y entonces la forma de tus errores depende de una versión que no controlas. Un record propio como cuerpo de error deja ese contrato en tus manos.

2 · La excepción dice qué pasó; el traductor decide cómo se cuenta.

La búsqueda lanza `ProductoNoEncontradoException` y no sabe nada de HTTP. El `@RestControllerAdvice` convierte eso en un 404 con cuerpo. Cada capa hace una cosa, y por eso el formato de todos los errores se cambia en un solo archivo.

3 · Validar es declarar la regla junto al dato — y encenderla.

Las anotaciones del record describen qué es válido, y `@Valid` es lo que las aplica. Sin `@Valid` no fallan: **aceptan**, que es la peor forma de fallar.

4 · Lo que se dice y lo que se apunta no son lo mismo.

El 500 genérico protege información que no debe salir; el `log.error` conserva lo que hace falta para arreglarlo. Quitar cualquiera de los dos rompe la mitad del trato.

5 Para profundizar

- **Pide una ruta que no existe** — `/productos/1/inventado` — y mira qué sale. ¿Lo recoge tu manejador de `NoResourceFoundException` o el genérico?
- **Manda un id que no sea un número**: `/productos/abc`. ¿Qué excepción es? ¿Te gusta lo que contesta? Escribe un manejador para ella.
- **Añade `@Size(max = 60)` al nombre** y comprueba que el mensaje sale en campos como los demás.
- **Quita el `@JsonInclude` del record de error** y vuelve a pedir un 404. Mira el "campos": `null` que aparece, y decide si te da igual.
- **Cambia el manejador genérico** para que devuelva `e.getMessage()` y pide otra vez la cuota con cero. Lee lo que sale. Después vuelve a dejarlo como estaba.

6 Antes de cerrar

Para la aplicación con `Ctrl+C`.

```
./mvnw clean
```

Lo que te llevas:

Las excepciones dicen qué pasó y no saben de HTTP. Un `@RestControllerAdvice` las traduce a respuestas con forma, en un solo sitio. Y lo que se le cuenta a quien llama no es lo mismo que lo que se apunta en el registro.

Lo que queda pendiente, y abre el Lab 04: todos los productos de este laboratorio viven en una lista dentro del código, y desaparecen al parar la aplicación. En el Lab 04 dejan de desaparecer.